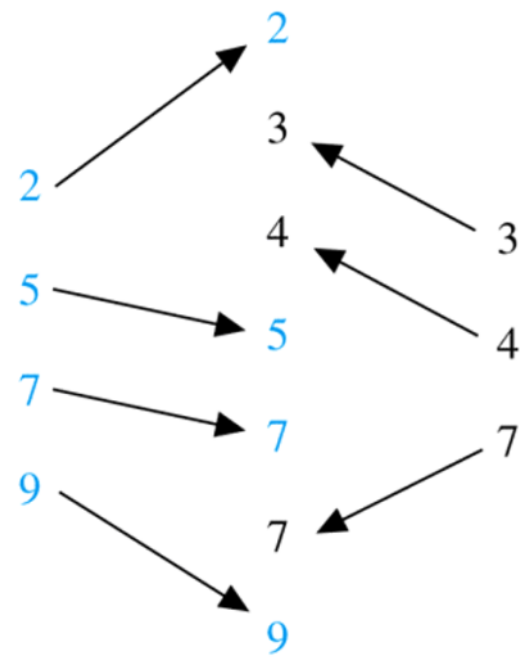

COMP 2121

DISCRETE

MATHEMATICS

Lecture 17



Merge Algorithm

Another application of Huffman's optimal binary tree algorithm arises in regard to the merging of sorted lists.

To merge together two sorted lists L_1 and L_2 into a single sorted list L we perform the following algorithm:

Merge Algorithm

Step 1: Set L equal to the empty list.

Step 2: Compare the first elements in L_1 , L_2 . Remove the smaller of the two from the list it is in and place it at the end of L .

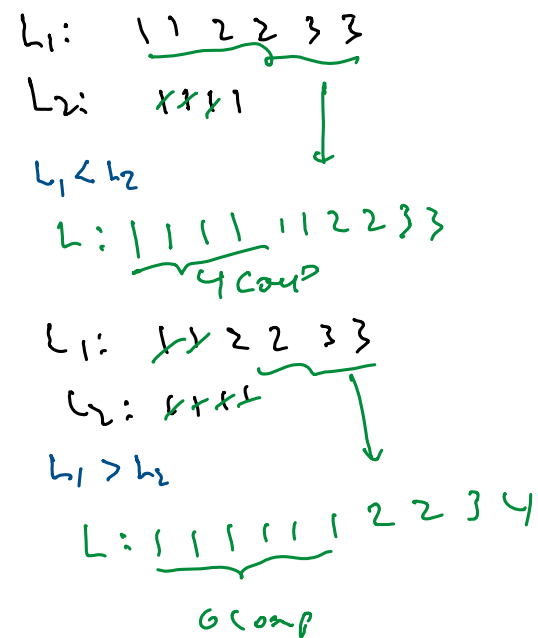
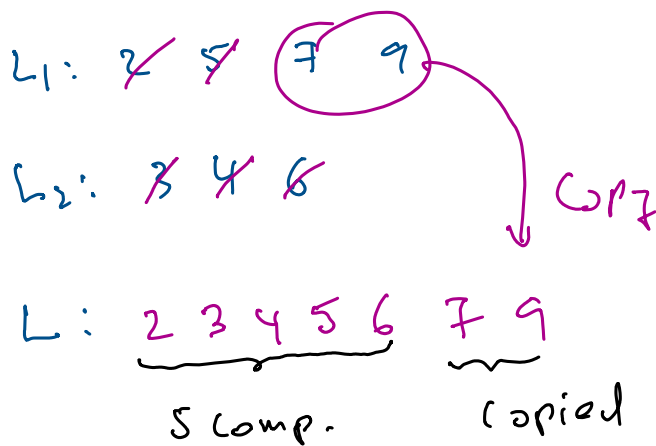
Step 3: If either of L_1 , L_2 is empty then

the other list is concatenated to the end of L

else go to step 2.

Example 1. Apply the Merge Algorithm to the two lists of numbers sorted in increasing order:

2, 5, 7, 9 and 3, 4, 6.



Example 2

Example 2. Give an example of two lists L_1 , L_2 , each of which is sorted in ascending order and contains five elements, and where

a) five comparisons are needed

b) nine comparisons are needed

to merge L_1 , L_2 by the Merge algorithm.

a) $L_1: 1, 2, 3, 4, 5$

$L_2: 6, 7, 8, 9, 10$ → copy

$L: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10$

 Comparisons

b) $L_1: 1, 3, 5, 7, 9$

$L_2: 2, 4, 6, 8, 10$ → copy

$L: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10$

 Comparisons

The maximum number of comparisons

Theorem. Let L_1, L_2 be two sorted lists of ascending numbers, where L_1 contains n_1 elements and L_2 contains n_2 elements. Then L_1 and L_2 can be merged into one ascending list L using at most $n_1 + n_2 - 1$ comparisons.

$L_1: a_1 a_2 a_3 \dots a_{n_1}$

$L_2: b_1 b_2 b_3 \dots b_{n_2}$

after several steps, in the worst case

$L_1: a_{n_1}$

$L_2: b_{n_2}$

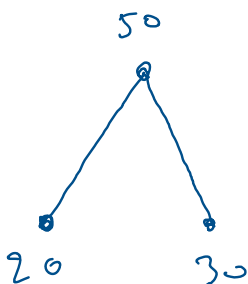
$L: \underbrace{\hspace{10em}}$
 $n_1 + n_2 - 2$

each element in the list L needed one compar. $n_1 + n_2 - 2$ of them

After comp. $a_{n_1} < b_{n_2}$ one list is empty.

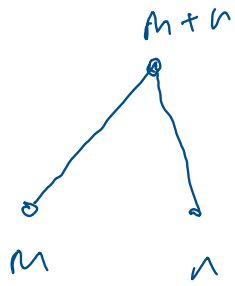
$\therefore$ Therefore in the worst case, the total number of comparisons is $n_1 + n_2 - 2 + 1 = n_1 + n_2 - 1$

At most $n_1 + n_2 - 1$ comparisons are needed.



at most: $20 + 30 - 1 = 49$
comparisons are needed.

at least: 20



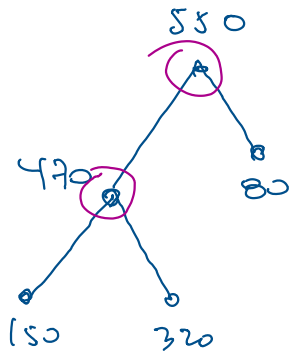
at most $m+n-1$

at least $\min(m, n)$

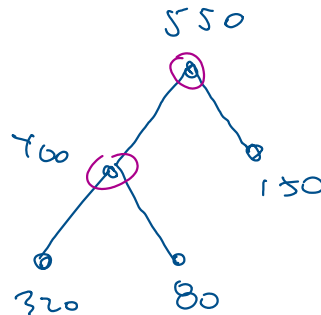
Comparing Merge Patterns

Example 3. Suppose we have three sorted lists L_1 , L_2 and L_3 containing 150, 320 and 80 numbers, respectively. By merging only two lists at the time, how can we merge these three lists into one sorted list so that we minimize the number of needed comparisons?

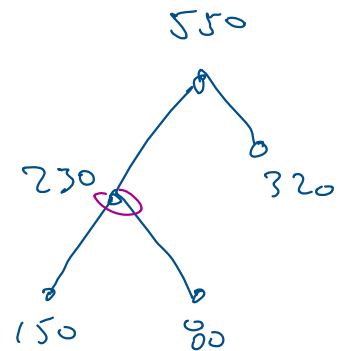
a) $(150 \& 320) \& 80$



b) $(320 \& 80) \& 150$



c) $(150 \& 80) \& 320$



at most

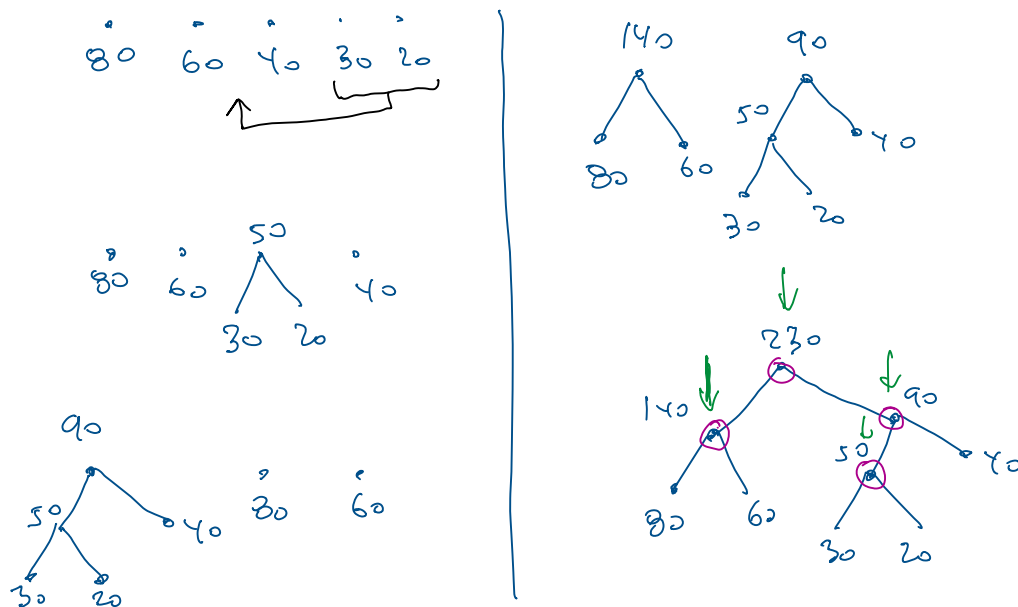
$$\begin{array}{r} 470 - 1 \\ + 550 - 1 \\ \hline = 1013 \end{array}$$

$$\begin{array}{r} 400 - 1 \\ + 550 - 1 \\ \hline = 948 \end{array}$$

$$\begin{array}{r} 230 - 1 \\ + 550 - 1 \\ \hline = 778 \end{array}$$

Optimal merge pattern

Example 4. We wish to merge five lists with 20, 30, 40, 60, and 80 numbers (each sorted in ascending order) into a single list. What is the optimal merge pattern? What is the maximum number of comparisons that the optimal merge pattern uses?



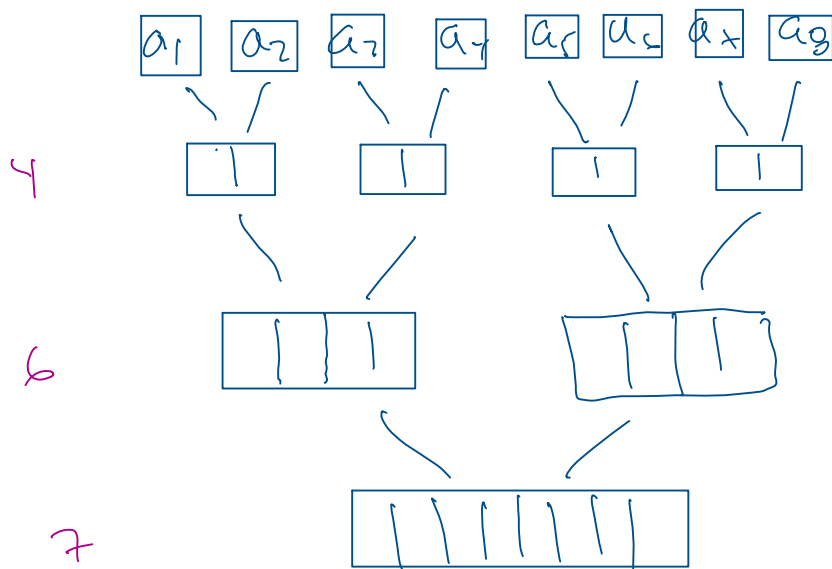
$$\text{at most: } (140-1) + (230-1) + (50-1) + (90-1) = 506$$

$$\text{at least: } 60 + 90 + 40 + 20 = \underline{\underline{210}}$$

MERGE SORT

$$-2 \quad 7 \quad 100 \quad -17 \quad 9 \quad 3 \quad 8 \quad 0$$

$$a_1 \quad a_2 \quad a_3 \quad a_4 \quad a_5 \quad a_6 \quad a_7 \quad a_8$$



$$2^3 = 8$$

$$3 = \log_2 8$$

$$\text{at most } 4 + 6 + 7 < 8 + 8 + 8 = 8(3) = 8 \log_2 8$$

$$n \log_2 n$$

$$O(n \log n)$$

Sort Comparison

Consider the following array:

26	4	14	37	31	20	32	34	30	9
---------------	--------------	---------------	---------------	---------------	---------------	---------------	---------------	----	---

a) Find the total number of comparisons needed to sort the array by using insertion sort – take the elements from the array one by one (starting at 26) and insert them in their correct position into a new sorted array. Finish the table below and find the total.

Step	New Array										# Comparisons
1	26										0
2	4	26									1
3	4	14	26								2
4	4	14	26	37							3
5	4	14	26	31	37						4
6	4	14	20	26	31	37					3
7	4	14	20	26	31	32	37				6
8	4	14	20	26	31	32	34	37			7
9	4	14	20	26	30	31	32	34	37		5
10	4	9	14	20	26	30	31	32	34	37	2

+ = 33

26	4	14	37	31	20	32	34	30	9
----	---	----	----	----	----	----	----	----	---

b) Find the number of comparisons needed to sort the array by using selection sort – find the minimum value and swap it with the first element, then repeat on the remaining part of the array. Finish the table below and find the total.

Step	Array										# Comparisons
0	26	4	14	37	31	20	32	34	30	9	-
1	4	26	14	37	31	20	32	34	30	9	9
2	4	9	14	37	31	20	32	34	30	26	8
3	4	9	14	37	31	20	32	34	30	26	7
4	:										6
5	:										5
6											4
7											3
8											2
9	4	9	14	20	26	30	31	32	34	37	1
10	4	9	14	20	26	30	31	32	34	37	0

$$\begin{aligned}
 &0 + 1 + 2 + \dots \\
 &\quad + 8 + 9 \\
 &= \frac{9(10)}{2} = 45
 \end{aligned}$$

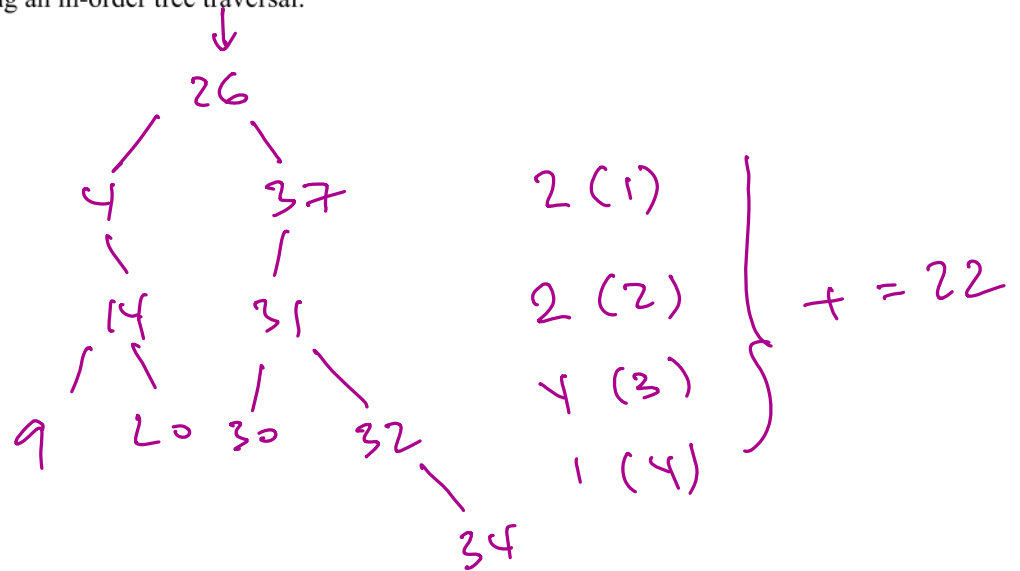
n - array

$$0 + 1 + 2 + \dots + (n-1) = \frac{(n-1)(n)}{2} = \frac{n^2}{2} - \frac{n}{2}$$

$$O(n^2)$$

26	4	14	37	31	20	32	34	30	9
----	---	----	----	----	----	----	----	----	---

c) Find the number of comparisons needed to sort the array by building a binary search tree and then doing an in-order tree traversal.



26	4	14	37	31	20	32	34	30	9
----	---	----	----	----	----	----	----	---------------	--------------

d) Find the number of comparisons needed to sort the array using merge sort.

4

26

4

14

37

31

20

32

34

5

4	26
--------------	----

14	37
---------------	----

20	31
---------------	---------------

32	34
----	----

7
+ = 16

4	14	26	37
--------------	---------------	---------------	----

20	31	32	34
---------------	---------------	---------------	---------------

4	14	20	26	31	32	34	37
---	----	----	----	----	----	----	----